

<table><tr><th colspan="2">Table 2</th></tr><tr><td>Irene Blanco</td><td rowspan="3">William Nimtz</td></tr><tr><td>Casallas</td></tr><tr><td></td></tr><tr><td></td><td>Sujal Vajire</td></tr></table>	Table 2		Irene Blanco	William Nimtz	Casallas			Sujal Vajire	<table><tr><th colspan="2">Table 3</th></tr><tr><td>Max Hortop</td><td>Lautaro Garcia</td></tr><tr><td></td><td></td></tr><tr><td>Dashiel Matlock</td><td>Lin Tian</td></tr></table>	Table 3		Max Hortop	Lautaro Garcia			Dashiel Matlock	Lin Tian	<table><tr><th colspan="2">Table 4</th></tr><tr><td>Jessica Duran</td><td>Ethan Kepros</td></tr><tr><td></td><td>Mohamed</td></tr><tr><td></td><td>Abdulazim Ali Afifi El</td></tr><tr><td>Mi Zhou</td><td>Habet</td></tr></table>	Table 4		Jessica Duran	Ethan Kepros		Mohamed		Abdulazim Ali Afifi El	Mi Zhou	Habet	<table><tr><th colspan="2">Table 5</th></tr><tr><td>Sam Hu</td><td>Jack Bajcz</td></tr><tr><td></td><td></td></tr><tr><td>Basma AlMahmood</td><td>Samuel Goodluck</td></tr></table>	Table 5		Sam Hu	Jack Bajcz			Basma AlMahmood	Samuel Goodluck
Table 2																																					
Irene Blanco	William Nimtz																																				
Casallas																																					
	Sujal Vajire																																				
Table 3																																					
Max Hortop	Lautaro Garcia																																				
Dashiel Matlock	Lin Tian																																				
Table 4																																					
Jessica Duran	Ethan Kepros																																				
	Mohamed																																				
	Abdulazim Ali Afifi El																																				
Mi Zhou	Habet																																				
Table 5																																					
Sam Hu	Jack Bajcz																																				
Basma AlMahmood	Samuel Goodluck																																				
<table><tr><th colspan="2">Table 1</th></tr><tr><td>Abraham Rai</td><td>Alexander Lover</td></tr><tr><td></td><td></td></tr><tr><td>Xin Dong</td><td>Jingyi Ge</td></tr></table>	Table 1		Abraham Rai	Alexander Lover			Xin Dong	Jingyi Ge		<table><tr><th colspan="2">Table 7</th></tr><tr><td>Saksham Mamtani</td><td>Andrew Tran</td></tr><tr><td></td><td></td></tr><tr><td>Sanjida Meherin</td><td>Aidana Bekbulatkyzy</td></tr></table>	Table 7		Saksham Mamtani	Andrew Tran			Sanjida Meherin	Aidana Bekbulatkyzy	<table><tr><th colspan="2">Table 6</th></tr><tr><td>Madison Mercer</td><td>Md Shariful Islam</td></tr><tr><td></td><td></td></tr><tr><td>Grace Akintan</td><td>Michael Pittman</td></tr></table>	Table 6		Madison Mercer	Md Shariful Islam			Grace Akintan	Michael Pittman										
Table 1																																					
Abraham Rai	Alexander Lover																																				
Xin Dong	Jingyi Ge																																				
Table 7																																					
Saksham Mamtani	Andrew Tran																																				
Sanjida Meherin	Aidana Bekbulatkyzy																																				
Table 6																																					
Madison Mercer	Md Shariful Islam																																				
Grace Akintan	Michael Pittman																																				

CMSE 801 - Day 6

Plotting models with matplotlib and using NumPy to do more complex math

Jan 29, 2026



General Course Announcements

- Reminder: **Homework 1 is due Feb 21.**
- Don't forget that we have office hours for you to stop by and get help or talk through the things we're learning! Check the [website](#).
 - If the scheduled times don't work for you, we can schedule by appointment.
- You should have access to the HiTA "AI Assistant" now
 - You are encouraged to experiment with the tool to explore how it can support your learning process in CMSE 801.

Questions/comments from Day 6 PCA

- generally find `matplotlib` challenging
 - getting the string right for specifying the style → check out [doc](#), scroll down to format strings. Or set different properties separately.
 - documentations not helpful
 - getting the right color
 - labels and legends
- drawing sine and cosine
- importing

Generating a set of finer-grained x values and plotting sine and cosine

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:

```
# Create a list from 0 to 2pi with increments of 0.1
x_list = []
x = 0.0
dx = 0.1
while x < 2*np.pi:
    x_list.append(x)
    x += dx

print(x_list)
```

```
[0.0, 0.1, 0.2, 0.30000000000000004, 0.4, 0.5, 0.6, 0.7, 0.7999999999999999, 0.8999999999999999,
0.9999999999999999, 1.0999999999999999, 1.2, 1.3, 1.4000000000000001, 1.5000000000000002, 1.600000
0000000003, 1.7000000000000004, 1.8000000000000005, 1.9000000000000006, 2.0000000000000004, 2.1000
0000000005, 2.2000000000000006, 2.3000000000000007, 2.4000000000000001, 2.5000000000000001, 2.6000
0000000001, 2.7000000000000001, 2.8000000000000001, 2.9000000000000012, 3.0000000000000013, 3.10000
00000000014, 3.2000000000000015, 3.3000000000000016, 3.4000000000000017, 3.5000000000000018, 3.600
00000000002, 3.7000000000000002, 3.8000000000000002, 3.9000000000000002, 4.0000000000000002, 4.100000
0000000001, 4.2000000000000001, 4.3000000000000001, 4.4, 4.5, 4.6, 4.699999999999999, 4.799999999999
99, 4.899999999999999, 4.999999999999998, 5.099999999999998, 5.199999999999975, 5.29999999999999
7, 5.399999999999997, 5.4999999999999964, 5.599999999999996, 5.699999999999996, 5.799999999999995,
5.899999999999995, 5.999999999999995, 6.099999999999994, 6.199999999999994]
```

In [3]:

```
# Calculate sine and cosine element-by-elements
sin_xlist = []
cos_xlist = []
for x in x_list:
    sin_xlist.append(np.sin(x))
    cos_xlist.append(np.cos(x))

print(sin_xlist)
print(cos_xlist)
```

```
[0.0, 0.09983341664682815, 0.19866933079506122, 0.2955202066613396, 0.3894183423086505, 0.47942553
8604203, 0.5646424733950354, 0.644217687237691, 0.7173560908995227, 0.7833269096274833, 0.84147098
48078964, 0.8912073600614353, 0.9320390859672263, 0.963558185417193, 0.9854497299884603, 0.9974949
866040544, 0.9995736030415051, 0.9916648104524686, 0.973847630878195, 0.9463000876874142, 0.909297
4268256815, 0.8632093666488735, 0.8084964038195899, 0.7457052121767197, 0.6754631805511503, 0.5984
721441039558, 0.5155013718214634, 0.42737988023382895, 0.33498815015590383, 0.23924932921398112,
0.1411200080598659, 0.04158066243328916, -0.05837414342758142, -0.15774569414324996, -0.2555411020
2683294, -0.3507832276896215, -0.44252044329485407, -0.5298361409084948, -0.6118578909427207, -0.6
877661591839753, -0.7568024953079294, -0.8182771110644114, -0.8715757724135886, -0.916165936749455
2, -0.9516020738895161, -0.977530117665097, -0.9936910036334644, -0.9999232575641008, -0.996164608
8358408, -0.9824526126243328, -0.958924274663139, -0.9258146823277331, -0.8834546557201545, -0.832
2674422239027, -0.7727644875559894, -0.7055403255703945, -0.6312666378723244, -0.5506855425976414,
-0.4646021794137613, -0.37387666483024096, -0.27941549819893097, -0.18216250427210112, -0.08308940
28175026]
[1.0, 0.9950041652780258, 0.9800665778412416, 0.955336489125606, 0.9210609940028851, 0.87758256189
03728, 0.8253356149096783, 0.7648421872844885, 0.6967067093471655, 0.6216099682706645, 0.540302305
8681398, 0.4535961214255775, 0.3623577544766736, 0.26749882862458735, 0.16996714290024081, 0.07073
720166770268, -0.029199522301289037, -0.12884449429552508, -0.22720209469308753, -0.32328956686350
396, -0.4161468365471428, -0.5048461045998579, -0.5885011172553463, -0.6662760212798247, -0.737393
7155412461, -0.8011436155469343, -0.8568887533689478, -0.9040721420170617, -0.9422223406686585, -
0.9709581651495908, -0.9899924966004456, -0.9991351502732795, -0.998294775794753, -0.9874797699088
647, -0.9667981925794605, -0.9364566872907957, -0.8967584163341462, -0.8481000317104072, -0.790967
7119144155, -0.7259323042001387, -0.6536436208636106, -0.5748239465332677, -0.49026082134069865, -
0.4007991720799746, -0.30733286997841935, -0.2107957994307797, -0.11215252693505487, -0.0123886634
62891447, 0.08749898343944551, 0.18651236942257401, 0.2836621854632246, 0.37797774271297857, 0.468
5166713003748, 0.5543743361791585, 0.6346928759426319, 0.7086697742912575, 0.7755658785102473, 0.8
347127848391573, 0.8855195169413168, 0.9274784307440339, 0.9601702866503645, 0.9832684384425836,
0.9965420970232169]
```

In [4]:

```
# Calculating it for the whole list as once (numpy let's us do this!)
sin_xlist = np.sin(x_list)
cos_xlist = np.cos(x_list)
print(sin_xlist)
print(cos_xlist)
```

```
[ 0.          0.09983342  0.19866933  0.29552021  0.38941834  0.47942554
  0.56464247  0.64421769  0.71735609  0.78332691  0.84147098  0.89120736
  0.93203909  0.96355819  0.98544973  0.99749499  0.9995736  0.99166481
  0.97384763  0.94630009  0.90929743  0.86320937  0.8084964  0.74570521
  0.67546318  0.59847214  0.51550137  0.42737988  0.33498815  0.23924933
  0.14112001  0.04158066 -0.05837414 -0.15774569 -0.2555411  -0.35078323
 -0.44252044 -0.52983614 -0.61185789 -0.68776616 -0.7568025  -0.81827711
 -0.87157577 -0.91616594 -0.95160207 -0.97753012 -0.993691  -0.99992326
 -0.99616461 -0.98245261 -0.95892427 -0.92581468 -0.88345466 -0.83226744
 -0.77276449 -0.70554033 -0.63126664 -0.55068554 -0.46460218 -0.37387666
 -0.2794155  -0.1821625  -0.0830894 ]
[ 1.          0.99500417  0.98006658  0.95533649  0.92106099  0.87758256
  0.82533561  0.76484219  0.69670671  0.62160997  0.54030231  0.45359612
  0.36235775  0.26749883  0.16996714  0.0707372  -0.02919952 -0.12884449
 -0.22720209 -0.32328957 -0.41614684 -0.5048461  -0.58850112 -0.66627602
 -0.73739372 -0.80114362 -0.85688875 -0.90407214 -0.94222234 -0.97095817
 -0.9899925  -0.99913515 -0.99829478 -0.98747977 -0.96679819 -0.93645669
 -0.89675842 -0.84810003 -0.79096771 -0.7259323  -0.65364362 -0.57482395
 -0.49026082 -0.40079917 -0.30733287 -0.2107958  -0.11215253 -0.01238866
  0.08749898  0.18651237  0.28366219  0.37797774  0.46851667  0.55437434
  0.63469288  0.70866977  0.77556588  0.83471278  0.88551952  0.92747843
  0.96017029  0.98326844  0.9965421 ]
```

In [5]:

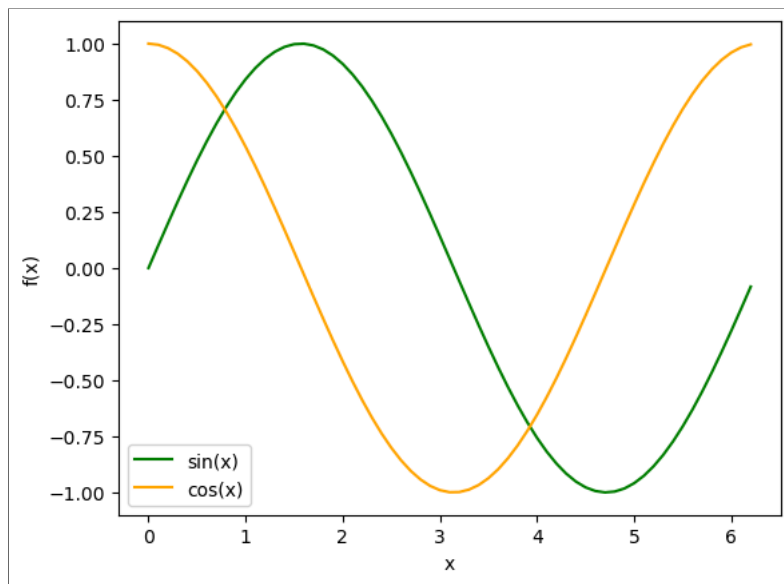
```
# I could have used a NumPy function to create the x values (you'll see this in the next PCA)  
dx = 0.1  
x_list = np.arange(0., 2.0 * np.pi, dx)  
print(x_list)
```

```
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.  1.1 1.2 1.3 1.4 1.5 1.6 1.7  
1.8 1.9 2.  2.1 2.2 2.3 2.4 2.5 2.6 2.7 2.8 2.9 3.  3.1 3.2 3.3 3.4 3.5  
3.6 3.7 3.8 3.9 4.  4.1 4.2 4.3 4.4 4.5 4.6 4.7 4.8 4.9 5.  5.1 5.2 5.3  
5.4 5.5 5.6 5.7 5.8 5.9 6.  6.1 6.2]
```


Now let's do some plotting

In [11]:

```
# Plotting
plt.plot(x_list, sin_xlist, "g", label='sin(x)') # use format string
plt.plot(x_list, cos_xlist, color="orange", label="cos(x)") # or use property
plt.legend()
plt.xlabel("x")
plt.ylabel("f(x)");
```

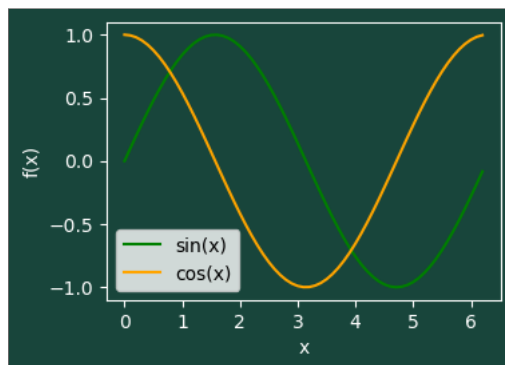


ONE CAN ALSO GET REALLY CARRIED AWAY WITH CUSTOMIZING THE PLOTS

In [13]:

```
# Define some parameters for the plot to customize it, note that this changes the defaults for all future plots
params = {"ytick.color" : "w",
          "xtick.color" : "w",
          "axes.labelcolor" : "w",
          "axes.edgecolor" : "w",
          "axes.facecolor" : "#18453B",
          "legend.facecolor" : "w"}
plt.rcParams.update(params)

# # Plotting
plt.figure(facecolor = "#18453B", figsize = (3.8, 2.7) )
plt.plot(x_list,sin_xlist,"g",label='sin(x)') # Note that a green line isn't maybe the best choice with our spartan-theme
plt.plot(x_list,cos_xlist,color="orange", label="cos(x)")
plt.legend()
plt.xlabel("x")
plt.ylabel("f(x)");
```



Goals for today

- Build a computational model for projecting population growth (using functions!)
- Test out a variety of parameters
- Visualize your model using plots in Python!

Looking forward - Day 7 PCA: Working more with NumPy and loading up some data

- More details on how NumPy can be one of the most useful modules/packages in Python
 - Arrays are super handy
- Try loading data for the first time!